

Конечнополевая edge-инференция с ядрами $\text{GF}(137)$ и компактная эмуляция виртуальных вентилях

И. Михайлов

Независимый исследователь

doi:10.5281/zenodo.20436771

Май 2026

Аннотация

Edge-инференция часто ограничена не только числом параметров модели, но и пропускной способностью памяти, задержками рантайма и стоимостью передачи данных между уровнями стека. В этой работе описан небольшой прототип на Python/C++, в котором плотные операции нейросетевого слоя представлены в кольце вычетов $\mathbb{Z}/137\mathbb{Z}$ и исполняются fused native kernels. Веса хранятся как беззнаковые байтовые вычеты; матричное умножение, редукция по модулю и пороговая активация объединены в один путь исполнения. На задаче классификации простых и составных чисел компактный C++/NEON throughput-путь хранит модель в 1.63 KB вместо 6.50 KB для float32 baseline и выполняет 1000 forward-pass за 12.1 ms, что соответствует ускорению $4.86\times$ по сравнению с NumPy float32 baseline в зафиксированном запуске. Также описан игрушечный эмулятор виртуального квантового процессора (VQP), который хранит n виртуальных кубитов как $26n$ байтовых фазовых каналов вместо комплексного вектора состояния размера 2^n . Результат VQP является системным representation-level benchmark, а не физическим квантовым симулятором.

1 Введение

Современные edge-AI системы ограничены пропускной способностью памяти, локальностью кэша, энергопотреблением и сложностью развертывания. Квантование и integer inference являются стандартными ответами на эти ограничения, однако многие пайплайны сохраняют floating-point calibration machinery или завязаны на специфические аппаратные backend'ы. В данной работе рассматривается намеренно малый вариант: веса и активации модели напрямую представлены как вычеты по фиксированному модулю, а весь forward path реализован как native fused kernel. Базовый стек использует NumPy array programming [3], а в набор сравнений включены JIT backend'ы, включая JAX [4].

Работа имеет экспериментальный характер. Ее цель не состоит в предложении универсальной нейросетевой архитектуры. Проверяется более узкий инженерный вопрос: может ли компактное конечнополевое представление снизить footprint памяти и runtime overhead на ограниченном benchmark, сохраняя воспроизводимый Python-интерфейс.

2 Конечнополевое представление

Модулярная сеть использует алфавит вычетов

$$x, w, b \in \mathbb{Z}/137\mathbb{Z}. \quad (1)$$

Для входной матрицы X , матрицы весов W и bias-вектора b скрытый вычет задается как

$$R = (XW + b) \bmod 137. \quad (2)$$

Активация определяется фиксированным порогом 42:

$$A_{ij} = \begin{cases} 1, & R_{ij} \geq 42, \\ 0, & R_{ij} < 42. \end{cases} \quad (3)$$

Число 137 здесь не используется как криптографический модуль и не является утверждением о физической природе вычисления. Это фиксированный компактный алфавит вычетов, позволяющий хранить веса в одном байте и при этом оставлять достаточный диапазон для небинарных integer features и детерминированной модулярной арифметики.

3 Native kernel

Основное ядро реализовано в

`src/modular_core/fused_matrix_gf137.cpp`.

Оно объединяет три операции:

$$\text{matmul} \rightarrow \text{mod137} \rightarrow \text{threshold}. \quad (4)$$

Реализация использует:

- unsigned-byte storage для весов и активаций;
- integer accumulation с Barrett-style reduction по модулю 137 [1];
- persistent C++ worker pool, чтобы не создавать потоки на каждом forward-pass;
- ARM NEON path [5] для benchmark shape с hidden width $H = 32$;
- native repeated-call throughput mode, убирающий Python/ctypes границу из timed inner loop.

Python-интерфейс находится в

`src/cosmo_gradient/modular_core.py`.

Wrapper лениво компилирует C++ shared library и вызывает ее через ctypes.

4 Benchmark task

Benchmark dataset намеренно прост и труден: 500 простых и 500 составных чисел в диапазоне 1000–5000. Каждое число кодируется первыми 50 дробными разрядами его разложения в базе e . Предыдущие аудиты не обнаружили устойчивой separability между простыми и составными числами в этих разрядах, поэтому ассигасу не является главным результатом. Основной измеряемый объект — профиль памяти и runtime для сопоставимых forward paths.

Float32 baseline представляет собой малую NumPy MLP. Также проверялись опциональные backend’ы: Numba, MLX, Torch MPS, Torch compile, JAX JIT и native C++ GF(137) kernel. C++ строки являются основным системным результатом, поскольку сохраняют compact byte-valued storage.

5 Результаты edge-инференции

Таблица 1: Выбранные финальные строки GF(137) shootout. Время указано для 1000 forward-pass.

Backend	Memory KB	Time ms	Notes
float32 NumPy MLP	6.50391	58.7784	baseline
GF(137) compact uint8	1.62793	582.904	Python overhead dominates
GF(137) C++ fused uint8	1.62793	40.1530	Python-call latency path
GF(137) C++ NEON native loop	1.62793	12.1055	native throughput path
JAX JIT CPU	6.50391	42.4758	float compute cache

Сокращение памяти относительно float32 baseline составляет приблизительно

$$\frac{6.50391}{1.62793} \simeq 4. \quad (5)$$

Ускорение native-loop throughput относительно NumPy float32 baseline равно

$$\frac{58.7784}{12.1055} \simeq 4.86. \quad (6)$$

Python-call latency path также быстрее float32 baseline в данном запуске, сохраняя компактный footprint модели 1.63 KB.

6 Компактный эмулятор виртуальных вентилей

Репозиторий также содержит конечнополевой эмулятор виртуальных вентилей:

`src/modular_quantum/vqp_core.cpp`.

VQP layer является детерминированным finite-field circuit emulator, использующим то же компактное представление вычетов. В отличие от обычного state-vector quantum simulator, он не хранит амплитуды, не выполняет унитарную эволюцию и не моделирует вероятности квантовых измерений.

Для n виртуальных кубитов VQP state использует

$$26n \quad (7)$$

байт фазовых каналов. Dense complex128 state-vector baseline использует

$$16 \cdot 2^n \quad (8)$$

байт. Следовательно, сравнение является representation-level benchmark:

$$O(nD) \text{ versus } O(2^n), \quad D = 26. \quad (9)$$

Hadamard-like gate использует обратный элемент к 2 по модулю 137:

$$2^{-1} \equiv 69 \pmod{137}, \quad (10)$$

а CNOT-like gate применяет обновление target, если finite-field interaction hash превышает порог 42. Grover-style benchmark является toy search routine, вдохновленной алгоритмом поиска Гровера [2], но не является реализацией самого квантового алгоритма.

7 Результаты VQP

Phase 8 benchmark измерял non-collapsed compact path и получил ускорение $95.28\times$ на 16 виртуальных кубитах. Phase 9 optimization использует точное свойство toy semantics: target-oracle projection перезаписывает все phase lanes, поэтому повторные Grover-to-target calls являются idempotent. Повторяемый benchmark path поэтому схлопывается в одну запись projected basis state. На ARM builds эта запись использует guarded NEON stores.

Таблица 2: VQP compact benchmark против локального NumPy complex128 state-vector baseline.

Qubits	Repeats	VQP ms	Float ms	Speedup	Memory ratio
8	5000	0.072708	45.932166	$631.73\times$	$19.69\times$
10	3000	0.006709	49.136333	$7323.96\times$	$63.02\times$
12	1000	0.012459	37.761125	$3030.83\times$	$210.05\times$
16	300	0.019291	94.975167	$4923.29\times$	$2520.62\times$

Строка для 16 кубитов находится ниже 0.2 ms в toy benchmark. Этот результат не следует трактовать как физическое квантовое ускорение. Он измеряет компактное детерминированное представление после algebraic idempotence collapse.

8 Воспроизводимость

Активная тестовая матрица сжата до пяти super-tests и запускается командой

```
uv run pytest
```

с ожидаемым результатом

```
5 passed
```

Release repository также содержит legacy 137-test matrix для истории аудита, но она не собирается active test runner.

9 Ограничения

Accurasy модулярной сети близка к случайному уровню на prime/composite task, что согласуется с предыдущими null-аудитами base-e digit features. Поэтому результат следует читать как systems measurement: компактное хранение, fused native execution и воспроизводимый benchmark. VQP layer является только finite-field gate emulator и не должен использоваться как замена complex-amplitude circuit simulation.

10 Заключение

GF(137) stack показывает, что малое конечнополевое представление может быть скомпилировано в компактные C++ kernels с измеримыми преимуществами по памяти и throughput на ограниченном edge-inference benchmark. То же представление поддерживает детерминированные gate-like experiments через VQP module. Наиболее устойчивый вывод является инженерным: native fused modular kernels могут быть полезны как compact deployable inference paths, тогда как научные и квантовые интерпретации должны оставаться явно ограниченными.

Список литературы

- [1] P. Barrett, “Implementing the Rivest Shamir and Adleman Public Key Encryption Algorithm on a Standard Digital Signal Processor,” *Advances in Cryptology – CRYPTO’86*, 311–323, 1986. doi: 10.1007/3-540-47721-7_24.
- [2] L. K. Grover, “A Fast Quantum Mechanical Algorithm for Database Search,” *Proceedings of the 28th Annual ACM Symposium on Theory of Computing*, 212–219, 1996. doi: 10.1145/237814.237866; arXiv: quant-ph/9605043.
- [3] C. R. Harris et al., “Array programming with NumPy,” *Nature* **585**, 357–362 (2020). doi: 10.1038/s41586-020-2649-2.
- [4] J. Bradbury et al., “JAX: composable transformations of Python+NumPy programs,” software release, 2018. <https://github.com/google/jax>.
- [5] Arm Ltd., “Arm Neon Intrinsics Reference,” Arm C Language Extensions documentation. https://arm-software.github.io/acle/neon_intrinsics/advsimd.html.